

Auto-MQT: Prompt-Adaptive Visual Token Budgeting for Efficient Large Vision-Language Models

Chi-Chun Chen

ciqchen@ucdavis.edu

Samuel Huang

godhuang@ucdavis.edu

Danny Kuei

dkuei@ucdavis.edu

University of California, Davis

Abstract

We propose **Auto-MQT**, a lightweight router for MQT-LLaVA. MQT-LLaVA can answer questions with different numbers of visual tokens, such as 16, 64, 144, or 256 tokens. This makes inference more flexible, because simple questions may not need many visual tokens, while harder questions may need more image details. However, the token still needs to be chosen manually at inference time. Our project studies whether a small router can choose this token budget automatically for each image-question pair. The router uses both the image representation and the user question to predict how many visual tokens are needed before answer generation. We keep the MQT-LLaVA backbone frozen and only train the router. To create training labels, we run the same example under multiple token budgets and use the smallest budget that gives a correct answer as the target. We will evaluate Auto-MQT by comparing answer accuracy, average visual token usage, and inference cost against fixed-budget baselines.

1 Introduction

Large vision-language models (LVLMs) are powerful, but they are also expensive because every image is turned into a long sequence of visual tokens that must be processed by the language model. In common architectures, this token count is fixed for every input, even though not all queries need the same amount of visual detail. A question such as “Is the person smiling?” may only require a coarse understanding of the image, while OCR-heavy or counting questions often need finer detail. Using the same visual token budget for both cases wastes computation on easy examples and makes deployment harder on limited hardware.

The Matryoshka Query Transformer (MQT) is a strong starting point for solving this problem Hu et al. showed that one model can support many inference-time token budgets by training a nested

visual tokenization scheme, and that some tasks remain strong with extremely small budgets (Hu et al., 2024). This is an important efficiency result, but it leaves a key decision unresolved: *how should the model decide which budget to use for a new image and prompt?* In the current form, MQT exposes flexibility, but it does not automate budget selection.

Our proposal is to add a small **Auto-MQT router** that observes the user prompt and a compact image representation, then predicts how many MQT visual tokens are needed before answer generation. The main scientific question is whether a tiny learned policy can recover most of the benefit of an oracle budget selector while keeping the backbone frozen. If successful, the project would turn MQT from a manually tunable mechanism into an adaptive inference system that better matches compute to instance difficulty.

This project is not mainly about making the model smaller. Instead, Auto-MQT focuses on deciding how much computation is needed for each image and prompt. The router learns to choose the number of visual tokens before the model generates an answer. In this way, the model can save computation while keeping good accuracy, and the original LVLM backbone can remain frozen.

2 Problem Definition and Method Design

2.1 Problem Definition

Let x be an image and q be a user question or instruction. A pretrained MQT-LLaVA model produces an answer distribution $p_\theta(y \mid x, q, m)$ using a selectable number of visual tokens $m \in \mathcal{M}$, where $\mathcal{M} = \{2, 4, 8, 16, 36, 64, 144, 256\}$ follows the budgets studied in the original paper (Hu et al., 2024). Larger m usually improves fine-grained perception but increases latency and compute. Smaller m is cheaper but may lose detail.

Our goal is to learn a routing function

$r_\phi(x, q) \rightarrow m$ that predicts the *smallest sufficient* token budget for each image-prompt pair. “Sufficient” means that the resulting answer quality is close to the best available quality under the same frozen backbone, while using fewer visual tokens. In other words, we want Auto-MQT to minimize compute without causing a large accuracy drop relative to a strong fixed-budget reference.

This leads to two concrete research questions. First, can prompt-image difficulty be predicted well enough to choose budgets adaptively? Second, does multimodal routing outperform simpler alternatives such as always using a fixed budget or predicting the budget from the prompt alone? Our hypothesis is that both the image and the question matter: OCR or counting prompts may demand more detail, but the same prompt can be easy for one image and hard for another.

2.2 Method Design

We will build Auto-MQT on top of the public **MQT-LLaVA-7B** introduced by Hu et al. (Hu et al., 2024). The 7B backbone will remain frozen throughout the project.

Step 1: Oracle budget labeling. For each training example, we will run the frozen MQT backbone on the candidate budgets in $\mathcal{M}$. We then identify the smallest budget m^* that is acceptable for that example. We will consider two label definitions:

- **Correctness-based label:** the smallest budget that produces a correct answer under the task metric.
- **Tolerance-based label:** the smallest budget whose score is within a small margin δ of the full 256-token prediction.

These labels give us a target budget without modifying the backbone. The oracle selector produced by exhaustive search will also serve as an upper bound during evaluation.

Step 2: Router. The router will take two frozen features as input: a prompt embedding from the language side and a compact image embedding from the vision encoder. We will fuse them with a very small network, such as a two-layer MLP or shallow cross-attention block, and output a categorical distribution over token budgets. The router is intended to be tiny compared with the 7B backbone, ideally under 10M trainable parameters, so that its cost is negligible relative to LVLM decoding.

We will evaluate two router variants. The first is a simple **late-fusion MLP** that concatenates pooled prompt and image features, which is the cheapest option. The second is a **tiny cross-attention router** in which a small number of prompt tokens attend to a compressed image summary before classification. This lets us test whether we actually need cross attention or if simple features are enough.

Step 3: Cost-aware training. Training the router only with standard cross-entropy would treat all mistakes equally, but in our setting predicting 256 when 144 would have worked is much less harmful than predicting 8 when 64 is necessary. We therefore propose a cost-aware objective

$$\mathcal{L} = \mathcal{L}_{\text{CE}}(m^*, \hat{m}) + \lambda \cdot \text{Cost}(\hat{m}) + \gamma \cdot \mathcal{L}_{\text{cal}},$$

where $\text{Cost}(\hat{m})$ is a normalized token or floating point operation (FLOP) cost, and $\mathcal{L}_{\text{cal}}$ encourages calibrated confidence so that the router does not become over-aggressive. We will tune λ on validation data to trade off efficiency and accuracy.

In addition to the hard target m^* , we will experiment with a **soft target distribution** over budgets derived from the observed accuracy-cost trade-off for each example. Intuitively, if both 36 and 64 tokens work well, the router should not be penalized as harshly for choosing either one as it would be for choosing 8. This variant better reflects the fact that budget quality is often near-tied across adjacent token counts.

Step 4: Inference policy. At test time, Auto-MQT first predicts a token budget, then the MQT-LLaVA model answers using only that many visual tokens. We will also consider a conservative fallback rule: if router confidence is low, use the next larger budget instead of the predicted one. This makes the system more robust and gives us a clean knob for controlling the risk-efficiency trade-off.

The core challenge is that MQT exposes a nested visual representation, but the correct budget depends on the joint semantics of the question and the image. Our method turns this into a supervised policy-learning problem over a discrete compute budget. It is lightweight, architecture-aware, and directly optimized for the scientific objective we care about: matching oracle-quality answers with lower average visual token usage.

We also expect the router to reveal interpretable structure about *when* visual detail matters. For example, prompts involving reading small text,

counting many objects, or localizing fine-grained attributes should prefer larger budgets, while high-level commonsense or binary questions may not. This interpretability angle is useful scientifically because it turns the project into more than a single accuracy number: it can explain how instance difficulty interacts with adaptive multimodal computation.

3 Related Work

Our proposal is most directly motivated by **MQT-LLaVA** (Hu et al., 2024), published at NeurIPS 2024. That paper introduces a Matryoshka query transformer so that one LVLm can operate with different numbers of visual tokens at inference time. Its main contribution is the *flexible nested tokenization mechanism*. However, the paper still assumes the token budget is chosen externally. Our work is complementary: we do not replace MQT, but instead add a learned policy that automatically selects a budget per instance.

Recent efficiency work has also studied **dynamic token pruning**. **MADTP** (Cao et al., 2024), published at CVPR 2024, adaptively prunes tokens in vision-language transformers using multimodal alignment guidance and instance-dependent compression ratios. This paper is important because it shows that dynamic multimodal compression is useful, but it focuses on earlier vision-language transformers rather than LVLms with nested query tokens. It also performs internal layer-wise pruning instead of choosing a single MQT budget before generation.

ATP-LLaVA (Ye et al., 2025a), published at CVPR 2025, moves closer to our setting by adaptively pruning vision tokens inside LLaVA decoder layers for each image-prompt pair. The method is instance-wise and layer-wise, which is highly relevant to our project. Still, ATP-LLaVA changes the decoder computation itself, whereas Auto-MQT uses the existing MQT nested structure and learns an external policy for choosing the minimum sufficient budget. This keeps the adaptation lightweight and backbone-compatible.

Another related direction is **learned visual compression**. **VoCo-LLaMA** (Ye et al., 2025b), published at CVPR 2025, trains LLM-aware compression tokens to reduce visual sequence length. This line of work improves compact representations, but it does not address prompt-adaptive budget selection on top of an already flexible model. In contrast,

our project studies decision-making over a discrete family of valid MQT budgets.

These papers together justify the novelty of Auto-MQT. Existing work has shown that (1) flexible token budgets are valuable (Hu et al., 2024), (2) dynamic compression can depend on the input instance (Cao et al., 2024; Ye et al., 2025a), and (3) learned compression modules can preserve performance (Ye et al., 2025b). What appears to be missing is a small, budget-selection router that exploits MQT’s nested tokenization to automatically choose *how much visual information to buy* for each image-question pair. That gap is exactly the focus of our project.

4 Evaluation Plan and Resources

We will evaluate both task quality and efficiency. The central question is whether Auto-MQT achieves a better accuracy-compute trade-off than fixed-budget MQT baselines under the same frozen backbone.

4.1 Datasets

We plan to use four standard image-question answering benchmarks that test out different visual and reasoning requirements, all of which are available on **HuggingFace**.

- **VQAv2**: A large-scale benchmark for general visual questions. These include yes or no questions, open ended questions, or questions asking to identify an amount.
- **GQA**: A benchmark used for visual and spatial reasoning. An example could be "What is the color of the fruit to the right of the apple?"
- **TextVQA**: A benchmark for visual reasoning based on text in images. An example could be a picture of some athletes with the question: "What is the name of the athlete on the left".
- **ScienceQA-IMG**: multimodal science questions with accompanying images. MQT reports that some reasoning-heavy tasks remain strong with fewer visual tokens, so this dataset is useful for testing whether Auto-MQT can exploit that structure automatically.

Together, these datasets provide coverage across a diverse set of questions. To keep the study executable within the quarter, we will likely cap router training to a subset of labeled examples per dataset

rather than exhaustively process all training sample. A practical plan is to use a larger subset for VQAv2 and GQA, where diversity is high, and smaller subsets for TextVQA and ScienceQA-IMG. If necessary, we could also consider reducing the number of benchmarks.

4.2 Evaluation Protocol

We will keep the MQT-LLaVA backbone, prompt template, and decoding setup fixed across methods so that the comparison isolates the effect of routing. Training is conducted using official training splits, hyperparameters will be selected on validation performance, and final numbers will be reported on held-out validation or test-dev splits depending on benchmark availability.

Baselines. We compare against the following baselines:

- **Fixed-budget MQT:** always use one budget from $\{8, 16, 36, 64, 144, 256\}$.
- **Prompt-only router:** predicts the budget from the question text without access to the image.
- **Image-only router:** predicts the budget from the image representation alone.
- **Oracle selector:** performs an exhaustive search over budgets to find the smallest acceptable one. This is an upper bound, not a deployable baseline.

Metrics. For task quality, we will use the standard metric for each dataset: VQAv2 accuracy, GQA accuracy, TextVQA accuracy, and ScienceQA accuracy. For efficiency, we will report average visual tokens per example, estimated visual-token FLOPs, and measured end-to-end latency on the same hardware. To summarize the trade-off, we will also report accuracy at matched average token budgets and the gap between Auto-MQT and the oracle selector.

We will also report two router-specific diagnostics. The first is budget regret which is the number of extra tokens used by Auto-MQT relative to the oracle minimum sufficient budget. The second is an under-budget rate, which measures how often the router selects too few tokens and loses answer quality. These diagnostics help distinguish whether the model is under allocating or over allocating resources to solve a task.

Fair comparison. Dynamic methods can be misleading if they are only compared at different compute levels. To avoid this, we will compare Auto-MQT against fixed-budget baselines under *matched average token budgets*. For example, if Auto-MQT averages about 36 tokens on a dataset, we will compare its accuracy to fixed 36-token MQT. This makes the evaluation fair because all methods use the same backbone and nearly the same compute budget.

Ablations. We plan to consider three ablations: (1) prompt-only versus image-only versus multi-modal routing; (2) different supervision rules for the oracle label; and (3) different candidate budget sets, such as coarse budgets $\{8, 32, 128, 256\}$ versus the full MQT set. These isolate which design choices matter most.

Beyond aggregate metrics, we will perform category-level analysis whenever dataset annotations allow it. In particular, we want to test whether Auto-MQT assigns larger budgets to counting and more open ended questions, and smaller budgets to coarse reasoning or yes/no questions. This analysis is important because a routing method that only improves the average could still fail systematically on visually demanding categories. Showing a sensible per-category pattern would support the claim that the router has learned an adaptive compute policy rather than memorizing a few dataset priors.

4.3 Estimated Computing Resources

The project is designed to be doable within the remaining course timeline because we are not re-training a 7B LVLM from scratch. Instead, we use a pretrained MQT-LLaVA-7B backbone. The only trainable component is the lightweight router, which should require a small fraction of the memory and time of full model tuning.

The most expensive stage is label generation, since each training example may need to be evaluated at multiple candidate budgets. We will reduce this cost in multiple ways such as stopping the search early once a sufficiently small correct budget is found or training the router on a stratified subset rather than the full benchmark training set. Under this design, we expect that the project can run on a modern 24GB GPU (such as an RTX 4090) or an A100 GPU. Router training itself should take only a few hours once oracle labels are prepared, while data generation may take roughly one to two GPU-days depending on subset size.

An additional advantage of our design is that most experiments are parallel. Oracle labels for different examples can be generated independently, and router sweeps over hyperparameters are cheap because the trainable model is small. This makes it realistic to finish multiple ablations and matched-budget comparisons within the 6 week course timeline, rather than spending the entire time budget on a single large training run.

Overall, the computational requirements are reasonable for this six weeks course project. If resources are more limited than expected, we can reducing the number of training examples or candidate budgets while preserving the core project.

References

- Jianjian Cao, Peng Ye, Shengze Li, Chong Yu, Yansong Tang, Jiwen Lu, and Tao Chen. 2024. [Madtp: Multimodal alignment-guided dynamic token pruning for accelerating vision-language transformer](#). In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15710–15719.
- Wenbo Hu, Zi-Yi Dou, Liunian Harold Li, Amita Kamath, Nanyun Peng, and Kai-Wei Chang. 2024. [Mqt-llava: Matryoshka query transformer for large vision-language models](#). In *The 38th Conference on Neural Information Processing Systems (NeurIPS)*.
- Xubing Ye, Yukang Gan, Yixiao Ge, Xiao-Ping Zhang, and Yansong Tang. 2025a. [Atp-llava: Adaptive token pruning for large vision language models](#). In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 24972–24982.
- Xubing Ye, Yukang Gan, Xiaoke Huang, Yixiao Ge, and Yansong Tang. 2025b. [Voco-llama: Towards vision compression with large language models](#). In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 29836–29846.